Session 11: Zoo Animal Tracker in Kotlin

1.Objective:

- To understand **Object-Oriented Programming (OOP)** concepts in Kotlin such as **inheritance, abstract classes, and interfaces**.
- To simulate a **zoo system** where different animals exhibit their unique behaviors.
- To implement **polymorphism** and **reusable class structures** in Kotlin.

2.Theory:

1. **Abstract Class:**
    - An abstract class cannot be instantiated directly.
    - It can contain properties and abstract functions that must be implemented by subclasses.
2. **Inheritance:**
    - Allows a class (subclass) to inherit properties and methods from another class (superclass).
3. **Interface:**
    - Defines a contract (methods) that classes can implement.
    - Supports multiple implementations unlike single inheritance in classes.
4. **Polymorphism:**
    - Ability of objects to take multiple forms.
    - In this lab, a list of Animal objects can hold Lion, Elephant, and Parrot objects, and call their respective behaviors dynamically.

3.Tools/Software Required:

- **Kotlin IDE** or **IntelliJ IDEA Community Edition**
- Basic knowledge of Kotlin syntax

4.Experiment / Procedure:

Step 1: Create an abstract class Animal

```
abstract class Animal(val name: String) {
    abstract fun makeSound()
}
```

Step 2: Create subclasses Lion, Elephant, and Parrot

```
class Lion(name: String) : Animal(name), Feedable {
    override fun makeSound() {
        println("$name: Roar!")
    }

    fun hunt() {
        println("$name is hunting in the savannah.")
    }

    override fun feed(food: String) {
        println("$name eats $food.")
    }
}

class Elephant(name: String) : Animal(name) {
    override fun makeSound() {
        println("$name: Trumpet!")
    }

    fun sprayWater() {
        println("$name sprays water from trunk.")
    }
}

class Parrot(name: String) : Animal(name), Feedable {
    override fun makeSound() {
        println("$name: Chirp!")
    }

    fun mimic(words: String) {
        println("$name mimics: $words")
    }

    override fun feed(food: String) {
        println("$name eats $food.")
    }
}
```

Step 3: Create Feedable interface

```
interface Feedable {
    fun feed(food: String)
}
```

Step 4: Create a function to display the zoo

```
fun displayZoo(animals: List<Animal>) {
    println("--- Zoo Tracker ---")
    for (animal in animals) {
        animal.makeSound()
        // Check if animal is feedable
        if (animal is Feedable) {
            animal.feed("food")
        }
        // Call special behaviors
        when (animal) {
            is Lion -> animal.hunt()
            is Elephant -> animal.sprayWater()
            is Parrot -> animal.mimic("Hello!")
        }
    }
}
```

Step 5: Main function

```
fun main() {
    val zooAnimals = listOf(
        Lion("Leo"),
        Elephant("Dumbo"),
        Parrot("Polly")
    )

    displayZoo(zooAnimals)
}
```

Sample Output:

```
--- Zoo Tracker ---
Leo: Roar!
Leo eats food.
Leo is hunting in the savannah.
Dumbo: Trumpet!
Dumbo sprays water from trunk.
Polly: Chirp!
```

Polly eats food.
Polly mimics: Hello!

5.Observations:

- Animal abstract class provided a **common structure** for all animals.
- Subclasses implemented their **unique behaviors** using **inheritance** and **overriding**.
- Feedable interface allowed **optional behaviors** for feeding.
- Polymorphism allowed **dynamic calling of methods** using the list of Animal objects.

6.Conclusion:

- This lab successfully demonstrated **OOP concepts in Kotlin**.
- **Inheritance** helped to reuse common code.
- **Abstract classes** enforced implementation of makeSound() in all animals.
- **Interfaces** provided flexibility for optional behaviors like feeding.
- Polymorphism allowed managing multiple animal types in a single list dynamically.